COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/16/2026	**DATE SUBMITTED:**	9/17/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 Implementing Trees

1. PROCEDURE OUTPUT

CODE SNIPPET

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

int main(){

    BinaryTree<char>* root = new BinaryTree<char>('A');
    root->insertChildLeft('B');
    root->insertChildRight('C');

    root->getLeftChild()->insertChildLeft('D');
    root->getRightChild()->insertChildRight('E');
    root->getRightChild()->getRightChild()->insertChildRight('F');

    std::cout << "The root: " << root->getRootVal() << std::endl;
    std::cout << "The left child of root A is: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The left child of the left child root A is: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of root A is: " << root->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of the right child root A is: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of E is: " << root->getRightChild()->getRightChild()->getRightChild()->getRootVal() << std::endl;

    root->getRightChild()->setRootVal('V');
    std::cout << "The NEW right child of root A is: " << root->getRightChild()->getRootVal() << std::endl;

    //Test here
    root->getLeftChild()->setRootVal('Y');
    std::cout << "The NEW left child of root A is: " << root->getLeftChild()->getRootVal() << std::endl;

    return 0;
}
```

binaryTree.h

binaryTree.h

```h
#ifndef BINARYTREE_H
#define BINARYTREE_H
#include <iostream>


template <typename T>
class BinaryTree{
    private:
        T key;

        //pointers for left and right chld
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;

    public:
        //constructor
        BinaryTree(T rootObj);
        //insert a child on the left node
        void insertChildLeft(T newNode);
        void insertChildRight(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        void setRootVal(T newNode);
};

//constructor
template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
};

//insert child
template <typename T>
void BinaryTree<T>::insertChildLeft(T newNode){
    if(leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    } else {
        BinaryTree<T>* newnode = new BinaryTree<T>(newNode);
        newnode->leftChild = leftChild;
        leftChild = newnode;
    }
}
```

```h

template<typename T>
void BinaryTree<T>::insertChildRight(T newNode){
    if(rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    } else{
        BinaryTree<T>* newnode = new BinaryTree<T>(newNode);
        newnode->rightChild = rightChild;
        rightChild = newnode;
    }
}

template <typename T>
BinaryTree<T> *BinaryTree<T>::getLeftChild(){
    return leftChild;
};

template <typename T>
BinaryTree<T> *BinaryTree<T>::getRightChild(){
    return rightChild;
};

template <typename T>
T BinaryTree<T>::getRootVal(){
    return key;
};

template <typename T>
void BinaryTree<T>::setRootVal(T newNode){
    key = newNode;
};


#endif
```

OUTPUT

```
The root: A
The left child of root A is: B
The left child of the left child root A is: D
The right child of root A is: C
The right child of the right child root A is: E
The right child of E is: F
The NEW right child of root A is: V
The NEW left child of root A is: Y
```

DOCUMENTATION: Explaining the methods/functions of the BinaryTree class created.

Constructor

Code in BinaryTree.h:

```
29  //constructor
30  template <typename T>
31  BinaryTree<T>::BinaryTree(T rootObj){
32      key = rootObj;
33      leftChild = nullptr;
34      rightChild = nullptr;
35  };
```

The constructor creates a new BinaryTree node. It stores the given value in the key. The **leftChild** and **rightChild** are set to nullptr because the new node does not have any children yet.

insertChildLeft()

Code in BinaryTree.h:

```
37  //insert Child on the left node
38  template <typename T>
39  void BinaryTree<T>::insertChildLeft(T newNode){
40      if(leftChild == nullptr){
41          leftChild = new BinaryTree<T>(newNode);
42      } else {
43          BinaryTree<T>* newnode = new BinaryTree<T>(newNode);
44          newnode->leftChild = leftChild;
45          leftChild = newnode;
46      }
47  }
```

This function inserts a new node on the left side of the current node. If there is no left child, it directly creates the new node. If a left child already exists, it creates the new node and places the existing left child underneath it.

insertChildRight()	**Code in BinaryTree.h:**
	`//insert Child on the right node`
	`template<typename T>`
	`void BinaryTree<T>::insertChildRight(T newNode){`
	`  if(rightChild == nullptr){`
	`  rightChild = new BinaryTree<T>(newNode);`
	`  } else{`
	`  BinaryTree<T>* newnode = new BinaryTree<T>(newNode);`
	`  newnode->rightChild = rightChild;`
	`  rightChild = newnode;`
	`  }`
	`}`
	
	This function essentially does the exact same thing as **insertChildLeft()** but it inserts a new node on the right side of the current node. If there is no right child, the new node is directly added. If a right child already exists, the new node is inserted and the previous right child is placed underneath it.

getLeftChild()	**Code in BinaryTree.h:**
	`template <typename T>`
	`BinaryTree<T> *BinaryTree<T>::getLeftChild(){`
	`  return leftChild;`
	`};`
	
	This function will return the pointer of the left child of the current node. Through this function, the program is able to access the node that lies on the left side of the current node. For instance, if we use **root->getLeftChild()**, then we access **B** since **B** is the left child of **A**.

getRightChild()	**Code in BinaryTree.h:**
	`template <typename T>`
	`BinaryTree<T> *BinaryTree<T>::getLeftChild(){`
	`  return leftChild;`
	`};`
	
	This works the same as the **getLeftChild()** function; the function will return the pointer of the right child of the current node. Through this function, the program is able to access the node that lies on the right side of the current node. For instance, if we use **root->getRightChild()**, then we access **C** since **C** is the right child of **A**.

getRootVal()	**Code in BinaryTree.h:**
	`template <typename T>`
	`T BinaryTree<T>::getRootVal(){`
	`  return key;`
	`};`

	This function will return the value stored in the current node. Through this function, the program is able to access and retrieve the value of a specific node in the binary tree. The key variable contains the value of the current node, which is what the function returns. For example, if we use **root->getRootVal()**, then we get the value **A** since **A** is the value stored in the current root node. So by using this, if we invoke **root->getLeftChild()->getRootVal()**, then we will be accessing **B** first using **getLeftChild()** and then it will retrieve its value using **getRootVal()**, which will then gives us **B**.

setRootVal()	**Code in BinaryTree.h:**
	`78 template <typename T>`
	`79 void BinaryTree<T>::setRootVal(T newNode){`
	`80 key = newNode;`
	`81 };`
	`82`
	`83`
	This function changes the value stored in the current node. It assigns the new value to the key variable without creating or removing a node. For example, the test we did for the last task needed for the activity **root->getLeftChild()->setRootVal('Y')** where it changes the value of **B** to **Y** while keeping the same node and its children.

Conclusion:

In this seatwork that will be used for my future activity, I learned how a binary tree is created and how its nodes are connected using pointers. I will be honest; I still lack knowledge about pointers, especially when we are using the linked list approach for trees. Despite the struggles, I've also learned how the different functions of the **BinaryTree class** are used to insert, access, retrieve, and change nodes and their values. The activity helped me understand how functions like **getLeftChild()** and **getRightChild()** allowed me to move through the tree, while **getRootVal()** and **setRootVal()** were used to access and modify the values stored in each node. Overall, this activity has given me a better understanding of how binary trees work and has given me newfound knowledge of how a tree is structured in code and how it's used conceptually. As of now, I do have to strengthen my critical thinking skills when learning this type of data structure because my foundation of pointers and link list implementation isn't that solid yet. Nonetheless, I've still gained the basic knowledge of what needs to be done, how it operates, and the task needed to be accomplished.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Google Overview (Gemini) and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, formatting, summarization, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to explain the concepts even further and obtain a better understanding of the overall structure of the code. It was also for grammar correction, misconceptions and to correct wording. All the code was done during the lecture session.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.